Aven-mo Payments Integration — Product Requirements

Status	Draft for review
Date	2026-11-05
Apps	Aven-mo iOS (`ios/`) and Android (`android/`)
Services	vensmall-wallet, vensmall-payments
Feature flags (Statsig)	`app_payments_integration`, `app_payments_paypal`
Related	Split Party PRD (draft), Firestore schema

1. Summary

Today the Aven-mo app settles a Split Party by writing mock payments and mock balances straight into Firestore. This PRD connects both mobile apps to the two new payment microservices:

- **vensmall-wallet** (`GET /payment-methods`) supplies the list of payment methods a joinee can pay their share with.
- **vensmall-payments** (`POST /payments`) records each joinee → host payment when the host taps **Collect**.

PayPal is added as the first **external wallet**: it appears in the payment-method list and can be used to post a payment. The whole integration is gated by a **Statsig** feature gate in the app, so it can be turned on per user, per platform or in percentages, and turned off instantly without an app release. With the gate off, the app behaves exactly as it does today.

2. Problem

- **Payments do not go through the payment platform.** The app's settlement writes `transactions` and `balanceCents` directly in Firestore (Paying a party). The payment domain now lives in dedicated services (extracted from vensmallmonolith), and the app is not using them.
- **Joinees cannot choose how to pay.** Every share is taken from the mock balance. There is no notion of a card, bank account or external wallet.
- **No external wallets.** Users who pay friends with PayPal have no way to do so in Aven-mo, and neither service knows about PayPal yet.
- **No safe rollout path.** The app has no feature-flag system. Switching the money path of a live demo without a kill switch is risky.

3. Goals and non-goals

Goals

#	Goal
G1	Joinees pick a payment method from the list served by vensmall-wallet.
G2	The host's Collect posts one payment per joinee to vensmall-payments, using the joinee's chosen method.

#	Goal
G3	PayPal is listed as an external wallet and is accepted when posting a payment.
G4	The integration is behind a Statsig gate, evaluated on the phone, default **off**, with an instant kill switch.
G5	With the gate off, behaviour, data and UI are identical to today.
G6	iOS and Android behave the same (same flags, same states, same copy).

Non-goals

- Real money movement, real card/bank processing or a real PayPal account link (OAuth, approval redirect, PayPal SDK). PayPal is simulated in the POC like the other methods (see Future work).
- Adding, removing or editing payment methods in the app.
- Per-user payment methods (the wallet serves one fixed catalogue in the POC).
- Making the Pay/Request tab functional (it stays "Coming soon").
- Moving balances out of Firestore. The feed, Me tab and balances keep reading Firestore.
- Authentication between the app and the services (both are open Function URLs today; see Risks).

4. User scenarios

S1 — Joinee picks a card. Annie joins Maxi's party at Tony's. With the gate on, the join confirmation shows "Pay with" and the methods from the wallet. She picks *Visa •••• 4242*. When Maxi taps Collect, a payment of her share is posted with `pm_card_visa_4242`, and the feed shows the payment as today.

S2 — Joinee pays with PayPal. Christophe has `app_payments_paypal` on. He sees *PayPal* in the list with the PayPal mark, picks it, and his share is posted with `pm_paypal`. The party summary shows "via PayPal" next to his row.

S3 — PayPal off for this user. The wallet returns PayPal, but Annie's `app_payments_paypal` gate is off. The app hides PayPal from her list. If an old value had selected it, the app falls back to the default method.

S4 — Wallet unreachable. The list fails to load. The joinee sees an inline "Couldn't load payment methods" with Retry, and can still join; the pre-selected **Venmsmall balance** is used.

S5 — Payment fails at Collect. vensmall-payments answers an error for one joinee. The party stays open, nothing is written to Firestore, and the host sees which friend's payment failed and can retry Collect.

S6 — Gate off. Everything works exactly as today: no "Pay with" row, no calls to either service, settlement writes Firestore only.

5. Current state

Aven-mo app

- Split Party settlement is one Firestore transaction run by the host's app (schema: Paying a party): it creates `transactions/{partyId}_{joinee}`, moves `balanceCents` with `FieldValue.increment`, marks the party `paid` and deletes the join-code lock. The deterministic id makes a retried Collect safe.
- Money is integer cents everywhere (`amountCents`, `billCents`).
- No feature-flag SDK, no HTTP client to backend services (only the Demo reset endpoint). Config comes from `ios/Config/*.xcconfig` and Android `buildConfigField`s.
- Analytics are local log events (`Analytics.swift`, `LogcatAnalytics.kt`).

vensmall-wallet

- `GET /payment-methods` → `200 {"data": [{id, type, label}]}`, a fixed catalogue that mirrors the monolith exactly until cutover:

id	type	label
`pm_card_visa_4242`	`card`	Visa •••• 4242
`pm_bank_chase_6789`	`bank`	Chase •••• 6789
`pm_balance`	`balance`	Venmsmall balance

- `type` is an enum `card | bank | balance`. No PayPal, no per-user data.
- `POST /payments` answers `404` (not its domain).

vensmall-payments

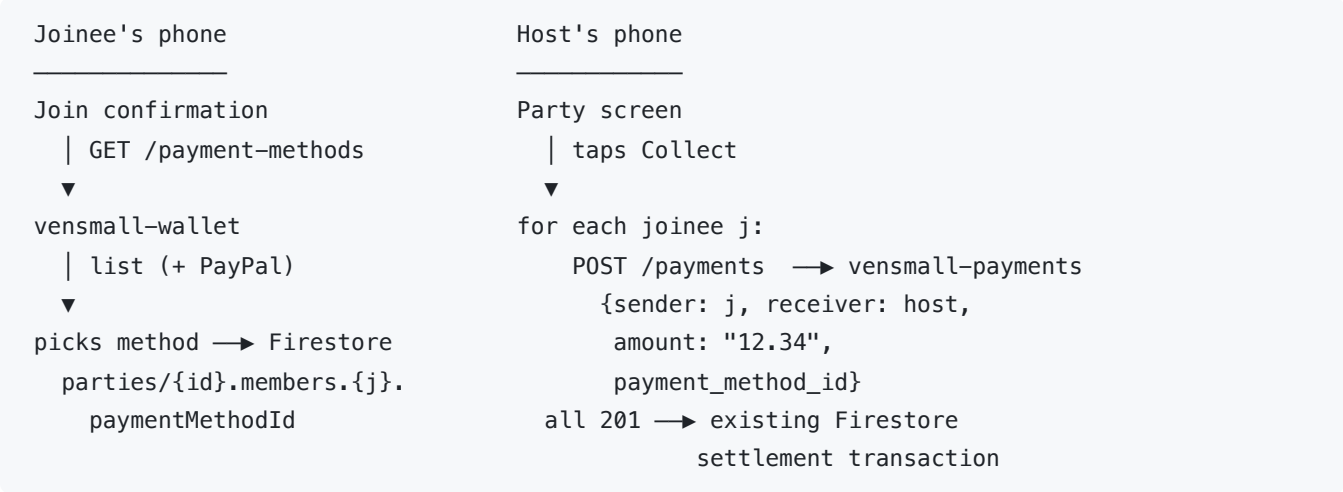
- `POST /payments` → `201` with `{id, sender, receiver, amount, payment_method_id, status, created_at}`. `amount` is a **decimal string** with ≤ 2 decimals; JSON numbers are rejected. `status` is always `completed`.
- Validates `payment_method_id` against its **own** fixed list (same three ids); it never calls the wallet. Unknown ids → `422 unknown_payment_method`.
- Errors: `422 {"errors": [{"field", "code", "message"}]}`.
- In-memory store only; no idempotency key.

Both services

Python + FastAPI on AWS Lambda behind open Function URLs (no auth, no API key, no CORS; concurrency capped as the only abuse guard). Every response carries `X-App-Version`.

6. Proposed solution

6.1 Overview



Gate **off** → the joinee path skips the wallet call and the "Pay with" row; the host path skips `POST /payments`. Only the existing Firestore transaction runs.

6.2 Listing payment methods (vensmall-wallet)

- The app calls `GET {WALLET_BASE_URL}/payment-methods` when the joinee reaches the join confirmation (and on Retry). Timeout 5 s.
- Contract change in the wallet (and, first, in the monolith, which is the source of truth until cutover):
  - New `type` value **`external_wallet`**.
  - New optional field **`provider`**, present only on external wallets.
  - New catalogue entry, appended last so existing order is unchanged:

```
{"id": "pm_paypal", "type": "external_wallet", "provider": "paypal", "label":
"PayPal"}
```

- The app **must ignore** entries whose `type` or `provider` it does not know, so the services can add methods without breaking shipped apps.
- The app hides `provider == "paypal"` entries when `app_payments_paypal` is off.
- Default selection: `pm_balance` if present, otherwise the first item.
- The joinee's choice is written to the party as `members.{uid}.paymentMethodId` (string). This is a schema addition to `firestore-schema.md` and `firestore.rules`; the field is optional so parties created with the gate off stay valid.

6.3 Posting payments (vensmall-payments)

- On Collect, the host's app posts one payment per joinee **before** the Firestore settlement:

```
POST {PAYMENTS_BASE_URL}/payments
Content-Type: application/json
Idempotency-Key: {partyId}_{joineeId}

{"sender": "annie", "receiver": "maxi", "amount": "26.22", "payment_method_id":
"pm_paypal"}
```

- `sender` / `receiver` are Aven-mo user ids (usernames, ≤ 20 chars, within the service's 64-char limit). `amount` is the joinee's share in cents formatted as a decimal string (`2622` → `"26.22"`), never a float.
- A joinee with no `paymentMethodId` (joined before the gate turned on, or the field is missing) is posted with `pm_balance`.
- The payments service adds `pm_paypal` to its accepted ids. Other services' validation is unchanged.
- **Idempotency:** the service must accept `Idempotency-Key` and return the original `201` for a repeated key, so a retried Collect never posts twice (today the Firestore id `{partyId}_{payerId}` gives this guarantee; the service must give the same one).
- **All or nothing from the user's point of view:** only when every POST returns `201` does the app run the existing Firestore transaction. If any POST fails, the party stays `open`, nothing is written to Firestore, and Collect can be retried; already-accepted payments are deduplicated by the idempotency key.
- The returned payment id is stored on the Firestore transaction as `paymentId` and the method as `paymentMethodId` (schema addition, optional fields), so the feed can show "via PayPal" and support can trace a payment across systems.

6.4 PayPal as external wallet

- POC behaviour: PayPal is a payment method like the others. The service answers `201 completed` synchronously; no redirect, no PayPal SDK.
- UI shows the PayPal name and mark (asset licensed per PayPal brand guidelines; see Open questions).

- The contract (`type: external_wallet`, `provider`) is designed so that a real PayPal flow (approval URL, `pending` status, webhook) can be added later without changing the list format.

6.5 Configuration

New per-environment settings, alongside the Firestore target:

Setting	iOS	Android
`WALLET_BASE_URL`	`ios/Config/*.xcconfig` → Info.plist	`buildConfigField`
`PAYMENTS_BASE_URL`	`ios/Config/*.xcconfig` → Info.plist	`buildConfigField`
`STATSIG_CLIENT_KEY`	`ios/Config/*.xcconfig` → Info.plist	`buildConfigField`

A missing URL or key means the integration is treated as **off**, whatever the gate says.

7. Feature flags

Statsig feature gates, evaluated client-side with the official Statsig iOS and Android SDKs.

Gate	Default	Controls
`app_payments_integration`	off	Master switch: wallet list on join, payments POST on Collect.
`app_payments_paypal`	off	Shows PayPal in the list. Effective only when `app_payments_integration` is also on.

Effective rules:

`app_payments_integration`	`app_payments_paypal`	Result
off	any	Today's behaviour. No service calls.
on	off	Wallet list without PayPal; payments posted.
on	on	Wallet list with PayPal; payments posted (PayPal allowed).

- **Statsig user:** `userID` = Aven-mo username; custom fields `platform` (`ios` / `android`) and `appVersion`. This allows targeting the three demo accounts, one platform, or a percentage.
- **When evaluated:** after sign-in (the SDK initialises with the user) and refreshed when the app returns to the foreground. Initialisation is bounded to 3 s; on timeout or error the SDK's cached values are used, and with no cache every gate is **off**.
- **Latched per party:** the host's app reads `app_payments_integration` once when the party is created and stores it as `paymentsIntegration: bool` on the party document. Joinees and Collect follow the party's value, not their own gate, so a party never mixes the two paths and a gate change mid-party has no effect on that party.
- **Kill switch:** turning `app_payments_integration` off in the Statsig console stops new parties from using the services at the next refresh.
- Exposure is logged by the SDK; gate checks happen only where the behaviour differs (join confirmation, party creation, Collect).

8. Functional requirements

ID	Requirement
FR-1	The app integrates the Statsig SDK on iOS and Android and initialises it after sign-in with the user described in §7.

ID	Requirement
FR-2	With `app_payments_integration` off (or not configured), the app makes no call to vensmall-wallet or vensmall-payments and writes no new fields.
FR-3	The host's app stores `paymentsIntegration` on the party at creation, from the gate value at that moment.
FR-4	On a party with `paymentsIntegration == true`, the join confirmation shows a "Pay with" selector populated from `GET /payment-methods`.
FR-5	The selector pre-selects `pm_balance` (else the first item) and lets the joinee change it until the party is collected.
FR-6	Entries with an unknown `type` or `provider` are not shown.
FR-7	PayPal (`provider == "paypal"`) is shown only when `app_payments_paypal` is on for the joinee.
FR-8	If the list fails to load, the selector shows an inline error with Retry; joining is not blocked and `pm_balance` is used.
FR-9	The joinee's choice is saved as `members.{uid}.paymentMethodId` on the party.
FR-10	On Collect, for a party with `paymentsIntegration == true`, the host's app posts one payment per joinee to `POST /payments` with sender = joinee, receiver = host, amount = share as a 2-decimal string, method = the joinee's `paymentMethodId` (default `pm_balance`), and `Idempotency-Key: {partyId}_{joineeId}`.
FR-11	Only when every POST returns `201` does the app run the existing Firestore settlement transaction, adding `paymentId` and `paymentMethodId` to each `transactions/{partyId}_{joinee}`.
FR-12	If any POST fails (network, timeout, 4xx, 5xx), the party stays `open`, no Firestore write happens, and the host sees which joinee failed and a Retry. A `422` shows the service's `message`.
FR-13	vensmall-wallet lists `pm_paypal` (`type: external_wallet`, `provider: paypal`, label "PayPal") last in the catalogue; the monolith's list is changed first and the wallet mirrors it.
FR-14	vensmall-payments accepts `pm_paypal` as a `payment_method_id`.
FR-15	vensmall-payments honours `Idempotency-Key`: a repeated key with the same body returns the original payment and `201`; a repeated key with a different body returns `409`.
FR-16	The party summary and the feed row show the method for party payments ("via PayPal", "via Visa •••• 4242") when `paymentMethodId` is present.
FR-17	iOS and Android implement FR-1 to FR-16 with the same states, copy and analytics events.

9. Non-functional requirements

ID	Area	Requirement
NFR-1	Latency	`GET /payment-methods` p95 ≤ 800 ms from the phone; Collect with up to 7 joinees completes in ≤ 3 s p95 (POSTs run in parallel).
NFR-2	Timeouts	5 s per HTTP call; Statsig init bounded to 3 s. No spinner without an end.
NFR-3	Money	Amounts stay integer cents in the app; conversion to the decimal string is exact (no floating point).
NFR-4	Safety	A retried Collect never creates a second payment (idempotency key + deterministic Firestore id).
NFR-5	Resilience	Statsig, wallet or payments being down never blocks join or a gate-off Collect.
NFR-6	Compatibility	Apps tolerate new fields and unknown method types; services keep the existing fields and error format.
NFR-7	Privacy	Only usernames and amounts are sent to the services; no device data. Statsig receives username, platform and app version only.
NFR-8	Observability	Each call logs the response `X-App-Version` and status in the app's analytics log.

ID	Area	Requirement
NFR-9	Quality	New code covered by unit tests on both platforms (HTTP clients with fakes, flag combinations, cents → string).

10. UX

- **Join confirmation (joinee), gate on:** below the share, a "Pay with" row shows the selected method (icon + label) and a chevron. Tapping opens a sheet listing the methods: card, bank, Venmsmall balance, then PayPal (when allowed). The selected row has a check mark.
- **Loading:** the row shows a skeleton for up to 5 s, then the selection.
- **Error:** "Couldn't load payment methods · Retry" inline under the row; the row shows *Venmsmall balance* as the fallback.
- **Party screen (host):** each member row shows "via {label}" under the share once the joinee has chosen. Collect is enabled under today's rules; the method choice never disables it.
- **Collect in progress:** the button shows a progress state; members are not editable.
- **Collect failed:** inline error "Couldn't collect from {name}. Nothing was paid." with Retry; the party stays open until it expires.
- **Gate off:** no visible difference from today.
- Visual details follow docs/design/visual-conventions.md; iOS and Android match.

11. Success metrics

Metric	Target (demo / pilot)
Collects with gate on that complete on the first try	≥ 95 %
Duplicate payments in vensmall-payments for one `{partyId}_{joinee}`	0
Joinees who see the method list without error	≥ 98 %
Share of gate-on joinees choosing a method other than the default	tracked (baseline)
Share of PayPal-enabled joinees choosing PayPal	tracked (baseline)
Time from gate flip to effect on new parties	≤ 1 app foreground

Analytics events (local log, same names on both platforms): `payment_methods_loaded`, `payment_methods_failed`, `payment_method_selected`, `collect_payments_posted`, `collect_payments_failed`.

12. Acceptance criteria

1. With both gates off, a full Split Party (start, join, bill, tip, Collect) behaves exactly as today and no request reaches either service.
2. With `app_payments_integration` on and `app_payments_paypal` off, the joinee sees card, bank and balance, not PayPal.
3. With both gates on, the joinee also sees PayPal, can select it, and the host's Collect posts a payment with `payment_method_id: "pm_paypal"` that returns `201`.
4. Each joinee's payment is posted once, with the share as a decimal string equal to the cents shown on screen.
5. Tapping Collect twice, or retrying after a timeout, creates one payment per joinee in vensmall-payments and one transaction per joinee in Firestore.

6. If one POST fails, the party stays open, no Firestore write happens, and the host sees the failure and can retry.
7. With the wallet unreachable, the joinee can still join and pays with Venmsmall balance.
8. Turning `app_payments_integration` off in Statsig makes the next party use today's path; a party already created keeps its `paymentsIntegration` value.
9. `GET /payment-methods` on the wallet and the monolith return the same body, including `pm_paypal` (`scripts/compare_with_monolith.sh` passes).
10. iOS and Android pass 1–8 with the same copy.

13. Risks

Risk	Impact	Mitigation
Services are open Function URLs with no auth	Anyone can post payments or read the list	POC only, mock money; concurrency caps; auth is a prerequisite before real money (Future work).
vensmall-payments stores payments in memory	Payments lost on cold start; idempotency lost between containers	Accept for POC; persistent store + idempotency table before pilot.
Partial failure across POSTs and Firestore	Service has a payment the feed does not show	Firestore write only after all `201`s; idempotent retries; party stays open for Retry.
Wallet and payments lists drift (`pm_paypal` in one only)	Listed method rejected with `422`	Ship payments change first, then monolith + wallet; contract test in both repos.
Statsig unavailable or slow at start	Wrong path or slow start	Bounded init, cached values, default off; gate latched per party.
Gate differs between host and joinees	Mixed behaviour in one party	Party-level `paymentsIntegration` decides; personal gate only affects PayPal visibility.
PayPal brand usage	Store rejection or legal issue	Use the official mark per PayPal guidelines; review before release.
Old app versions	Ignore new fields	Fields are optional; old apps keep today's path.

14. Dependencies

Dependency	Owner	Needed for
vensmallmonolith: add `pm_paypal` and `external_wallet` / `provider` to the payment-method list	Payments platform	Source of truth until cutover (FR-13)
vensmall-wallet: mirror the monolith's new catalogue and model	Wallet team	FR-13
vensmall-payments: accept `pm_paypal`; support `Idempotency-Key`	Payments team	FR-14, FR-15
Deployed wallet and payments URLs per environment	Payments platform	§6.5
Statsig project, client SDK key, the two gates created	Mobile / product	§7
Statsig iOS SDK (SPM) and Android SDK (Gradle)	Mobile	FR-1
Firestore schema + rules: `paymentsIntegration`, `members.{uid}.paymentMethodId`, `transactions.paymentId`, `transactions.paymentMethodId`	Mobile	FR-3, FR-9, FR-11

Dependency	Owner	Needed for
PayPal mark asset and brand approval	Design / legal	§10

15. Open questions

1. Should joinees choose the method at join time (proposed) or should each joinee confirm their own payment at Collect time?
2. Is `external_wallet` + `provider` the agreed wallet contract, or should PayPal be its own `type: "paypal"`?
3. Who owns the method catalogue after cutover: the wallet alone, with the monolith retired?
4. Should vensmall-payments validate methods by calling the wallet instead of keeping its own copy, now that the list grows?
5. Does `Idempotency-Key` live in vensmall-payments for the POC, or do we accept the risk until persistence lands?
6. Do balances keep moving in Firestore for card/bank/PayPal payments, or only for `pm_balance`?
7. Which Statsig environment(s) does the demo use, and who can flip the gates?

16. Future work

- Real PayPal: account linking, approval redirect, `pending` → `completed` status, webhooks, refunds.
- Per-user payment methods from the wallet (add/remove cards, link PayPal).
- More external wallets (Apple Pay, Google Pay) using the same `external_wallet` contract.
- Authentication between the apps and the services (signed user token).
- Persistent payment store and server-side settlement, so the phone no longer writes balances.
- Pay/Request tab using the same wallet and payments services.
- Remove `app_payments_integration` once fully rolled out and the Firestore-only path is retired.

17. References

- Split Party PRD (draft)
- Firestore schema
- Visual conventions
- vensmall-wallet — `GET /payment-methods`
- vensmall-payments — `POST /payments`
- vensmallmonolith — source of truth for the method list until cutover
- Statsig iOS SDK and Android SDK
- PayPal brand guidelines